COMPUTER SCIENCE + COMPUTER ENGINEERING

# Practice, build, and verify

Laboratory and module-project workbook

A place to turn the explanations into evidence: function exercises, manual workflows, changed inputs, integration projects and module reviews.

**20 lab briefs · 50 additional exercises · 25 review questions**

Package 02 · Expanded teaching edition · September 28, 2026
Volume 01 of the full undergraduate-to-graduate program
Self-study material. Completion requires implementation, testing and review.

# Contents

Use the linked contents or PDF bookmarks to navigate.

**Using the practical workbook** ..... 3

**Module 01: Starting as an engineer** ..... 4
- Lab 01.1: What a program actually does ..... 4
- Lab 01.2: Files, terminals and reproducible execution ..... 5
- Lab 01.3: Trace before running ..... 6
- Lab 01.4: Units, budgets and engineering notebooks ..... 7
- Module 01 integration project: An explainable motion experiment ..... 7

**Module 02: Values, decisions and repetition** ..... 9
- Lab 02.1: Types and expressions ..... 9
- Lab 02.2: Conditions and boundary cases ..... 10
- Lab 02.3: Loops and invariants ..... 11
- Lab 02.4: Functions and contracts ..... 12
- Module 02 integration project: A bounded five-turn resource game ..... 12

**Module 03: Organizing data and small programs** ..... 14
- Lab 03.1: Sequences, dictionaries and identity ..... 14
- Lab 03.2: Decomposition, modules and interfaces ..... 15
- Lab 03.3: Files, serialization and validation ..... 16
- Lab 03.4: Classes, composition and state machines ..... 17
- Module 03 integration project: A data-driven simulator with validated persistence ..... 17

**Module 04: Debugging, testing and collaboration** ..... 19
- Lab 04.1: Debugging as an experiment ..... 19
- Lab 04.2: Unit, integration and property tests ..... 20
- Lab 04.3: Git, branches and recoverable changes ..... 21
- Lab 04.4: Review, documentation and responsible AI help ..... 22
- Module 04 integration project: A reviewed, tested local release ..... 22

**Module 05: A complete first game** ..... 24
- Lab 05.1: The update loop and command model ..... 24
- Lab 05.2: Collision rules and complete behavior ..... 25
- Lab 05.3: Deterministic replay and simple measurement ..... 26
- Lab 05.4: Finishing, usability and the first portfolio artifact ..... 27
- Module 05 integration project: Training Yard: the Gate A first-game release ..... 27

# Using the practical workbook

Each lesson has one practical brief. Sixteen lessons also have function starters and a contract checker; four require environmental, Git, review or release evidence. The first-module function extensions should wait until 02.4. Complete a lesson's written reasoning even where a checker exists.

For an automated exercise, edit `labs/student/lab_MM_L.py`, then run `python3 tools/check_labs.py --lesson MM.L` from the package root using your verified launcher. The starter deliberately raises NotImplementedError. Reference answers are separate under labs/reference/. The checker covers selected contracts, not all possible behavior and not the learner's independent understanding.

At each module end, complete ten additional exercises, the five-question review and the integration project. Worked answers and assessment guidance are in ANSWER_KEY.pdf. Use the templates for lab records, bugs, benchmarks and reviews. Suggested project pass: 80/100 using the Gate A rubric, with all essential requirements met.

# Module 01: Starting as an engineer

**Prerequisites:** None. Basic reading, arithmetic and the ability to save a file.

## Lab 01.1: What a program actually does

**Goal:** Explain state, instructions, input and output using a tiny game.

Start on paper at x=2. Trace right 3, left 1, right 0, left 4, right 2. Include the initial state and all five following states. Mark which operations are reads and which are writes. Then run `examples/example_01_1.py` twice and compare its smaller demonstration with your prediction.

Create `workspace/01_1_trace.md`. Include the six-position trace, a plain-language update rule, and an example where two wrong movements cancel. Your pass evidence is an explanation of every transition and why drawing twice must not move twice. The optional function scaffold `labs/student/lab_01_1.py` is a return visit after Lesson 02.4; functions and loops are not prerequisites for this paper-first lesson.

**Changed input:** Change the initial location to -3 without changing the commands. Predict every state, not only the final value.

### Function interfaces

`trace_moves(start, deltas)`: Return the initial integer position and each following position.

**Timing:** complete this optional code wrapper after Lesson 02.4.

### Submission record

Prediction / expected case:

Actual result and command:

Failure case and repair:

Changed-case explanation:

Reviewer/self-check and remaining gap:

# Lab 01.2: Files, terminals and reproducible execution

**Goal:** Run a saved Python program and distinguish a shell from an interpreter.

Run `tools/environment_check.py` and save the output with your actual command in `workspace/01_2_environment.md`. Run the movement example, then deliberately change only its filename in the command to a name that does not exist. Record the resulting error, identify which program emitted it and repair the command.

Repeat the valid command from the package root. Your evidence must distinguish a script path from the working directory and explain how a relative filename is resolved. This is an evidence-based lab: no Python function starter is assigned and the automatic lab checker does not certify environment understanding.

**Changed input:** Run the example using its absolute path from a different working directory. Predict which reported field changes before executing.

**Check type:** evidence-based review, not an automated function test.

## Submission record

Prediction / expected case:

Actual result and command:

Failure case and repair:

Changed-case explanation:

Reviewer/self-check and remaining gap:

# Lab 01.3: Trace before running

**Goal:** Predict a program by following assignments in order.

Create three four-statement assignment sequences in `workspace/01_3_trace.md`. Include an incorrect swap, the repaired swap and a sequence in which a derived value does not update automatically. For each, predict every binding and final printed output before checking with Python.

The optional starter `labs/student/lab_01_3.py` packages the swap in a function; return to it after Lesson 02.4. Pass now by explaining where the incorrect swap loses the old value and by solving a fresh sequence without running it first. Do not count a memorized one-line swap as sufficient explanation.

**Changed input:** Use starting values -5 and 0 in your repaired swap. Then make a two-variable example whose behavior changes when two assignments are reversed.

## Function interfaces

`swap_values(a, b)`: Return the two input values in reversed order.

**Timing:** complete this optional code wrapper after Lesson 02.4.

## Submission record

Prediction / expected case:

Actual result and command:

Failure case and repair:

Changed-case explanation:

Reviewer/self-check and remaining gap:

# Lab 01.4: Units, budgets and engineering notebooks

**Goal:** Use units and simple arithmetic to check game calculations.

Calculate all six combinations of speeds 2, 6 and 10 units/second with intervals 0.1 and 0.5 seconds. Then calculate the frame intervals for 30, 60 and 120 frames/second. Show units and identify the chosen targets. Store the table in `workspace/01_4_units.md`.

After functions are introduced, implement the `displacement` and `frame_interval_ms` starter contracts. Their input domain is finite numeric values, with nonnegative elapsed time and positive frame rate; finite overflow results must also be rejected. These programmatic checks supplement rather than replace the written unit analysis.

**Changed input:** A movement program receives 16 milliseconds and a speed of 2.5 meters/second. Derive the displacement and compare it with the mistaken calculation using 16 seconds.

## Function interfaces

`displacement(speed, dt)`: Finite numeric speed; finite, nonnegative dt. Return displacement.

`frame_interval_ms(fps)`: Finite positive frames/second -> milliseconds/frame.

**Timing:** complete this optional code wrapper after Lesson 02.4.

## Submission record

Prediction / expected case:

Actual result and command:

Failure case and repair:

Changed-case explanation:

Reviewer/self-check and remaining gap:

# Module 01 integration project: An explainable motion experiment

Create a dedicated learning folder containing a working motion program, environment report, prediction table, unit calculations and a one-page explanation of input, state, update and output. This folder becomes a Git repository in Module 04; Git is not an undeclared prerequisite today.

Use the original five-command trace and two changed starting positions. The program must produce a trace that agrees with the independently prepared table. Record the exact command and interpreter. Include one controlled missing-file failure and one wrong-units example, then explain the repair. Do not change system settings or create public accounts to complete this project.

**Acceptance:** A reviewer can repeat the command, match the trace, identify the working directory and hear an explanation of why two renders do not cause two updates. Complete the first-module paper work before using the optional function starters after 02.4.

## Ten additional exercises

1. **M01-E01.** Trace start=7 with changes -2,+5,-8,+1. Include the initial state.
2. **M01-E02.** Construct two wrong intermediate movements that preserve the correct final position. Explain why final-only testing misses them.
3. **M01-E03.** Classify a keypress, player position, drawing function and monitor image as input, state, rule/operation or output.
4. **M01-E04.** From a parent folder, explain how to run a script in Game_Engineering_Package_02/ examples without changing folders.
5. **M01-E05.** Trace a=5; b=a+1; a=b+1; c=a+b.
6. **M01-E06.** Explain why swapping equal values is a weak test of a swap algorithm.
7. **M01-E07.** Convert 16 milliseconds to seconds and calculate travel at 2.5 units/second.
8. **M01-E08.** Calculate the target interval at 144 frames/second. Label it as a target or measurement.
9. **M01-E09.** Write a reproducibility recipe for one example using your actual environment.
10. **M01-E10.** Explain the distinction between negative velocity and negative elapsed time in this course.

## Five-question module review

Each answer earns 0-2 points. Suggested pass 8/10 plus project evidence.

1. **M01-R01.** Why can rendering the same world from two views expose a poor state/update boundary?
2. **M01-R02.** A script file exists but cannot be found with a relative path. What fact must be checked?
3. **M01-R03.** What are x and y after x=3; y=x+4; x=y*2?
4. **M01-R04.** At 30 frames/s, does a stated 40 ms serial frame fit the interval?
5. **M01-R05.** What evidence lets a reviewer distinguish a prediction from an observation?

**Project evidence:** source and inputs, test results, known limits, changed requirement, review decision.

# Module 02: Values, decisions and repetition

**Prerequisites:** Module 01.

## Lab 02.1: Types and expressions

**Goal:** Choose integers, real-valued numbers, strings and booleans appropriately.

Implement `total_cost(count_text, unit_cost_text)` in `labs/student/lab_02_1.py`. Use whole-number text accepted by Python's integer conversion; reject malformed text and negative values with `ValueError`. Return the product without printing. Test count zero, price zero, ordinary values, surrounding spaces, negative values and fractional text.

Run `tools/check_labs.py --lesson 02.1` with your selected Python launcher. Also create a small interaction script that asks for two strings, calls the function and catches only the expected input error. Save an input/output table. Passing the supplied checks is necessary for the function exercise, but you must explain parsing versus domain validation independently.

**Changed input:** Add a maximum allowed purchase count of 99 in a separate experiment. Test 98, 99 and 100 without changing the meaning of zero.

### Function interfaces

`total_cost(count_text, unit_cost_text)`: Convert whole-number text; reject negative quantities and bad text.

### Submission record

Prediction / expected case:

Actual result and command:

Failure case and repair:

Changed-case explanation:

Reviewer/self-check and remaining gap:

# Lab 02.2: Conditions and boundary cases

**Goal:** Write decisions with complete and non-overlapping rules.

Implement `can_open` in `labs/student/lab_02_2.py`. Prepare the 18-row table using key=false/true, level=4/5/6 and stamina=9/10/11. Add the key=true, level=1, stamina=0 counterexample. Explain why an expression requiring stamina for every key holder would fail that case.

Run the lesson checks and write one caller that prints an open/closed message based on the returned Boolean. Do not deduct stamina in the predicate. Pass by defending the grouping, the inclusive comparisons and every table row, not just by observing a green test run.

**Changed input:** Specify an alternative door rule in which using a key costs stamina. Write the new rule separately; do not silently change the original assignment.

## Function interfaces

`can_open(has_key, level, stamina)`: Trusted inputs: bool key and nonnegative integer level/stamina.

## Submission record

Prediction / expected case:

Actual result and command:

Failure case and repair:

Changed-case explanation:

Reviewer/self-check and remaining gap:

# Lab 02.3: Loops and invariants

**Goal:** Use repetition that terminates and preserves a stated property.

Implement `simulate_resources(turns)` in `labs/student/lab_02_3.py`. Return the initial zero plus the value after every complete turn. Reject negative, non-integer and Boolean turn counts. The strict Boolean exclusion is a schema choice: a yes/no flag is not an acceptable turn count even though Python relates bool and int.

Trace ten turns manually and compare all eleven values. Test zero turns. Explain the nonnegative invariant and show a changed example that breaks an unconditional-spending version. Run the supplied checks, then create at least two additional tests with turn counts not used in the demonstration.

**Changed input:** Change collection to seven units per turn in a separate experiment. Explain why a single optional subtraction no longer guarantees a final count below five.

## Function interfaces

`simulate_resources(turns)`: Start at zero; each turn collect 3, then spend 5 if affordable. Return initial state followed by one resource count per turn.

## Submission record

Prediction / expected case:

Actual result and command:

Failure case and repair:

Changed-case explanation:

Reviewer/self-check and remaining gap:

# Lab 02.4: Functions and contracts

**Goal:** Separate a calculation from interaction and state its valid inputs.

Implement the documented function in `labs/student/lab_02_4.py`. Test normal movement, zero velocity, negative velocity, zero dt, negative dt, infinity, not-a-number and an overflowing result. Capture or inspect output to confirm the function itself does not print.

Write a contract comment naming units and side effects. Then revisit the optional first-module function starters and run their checks. Pass by explaining the difference between returning and printing, and by showing that a caller can retain its old value after catching an invalid update request.

**Changed input:** Write a second function that converts milliseconds to seconds and calls advance. Keep unit conversion at this boundary and reuse the original movement function unchanged.

## Function interfaces

`advance(position, velocity, dt)`: Finite numeric inputs; dt >= 0; return a finite new position. Does not print, read input or change external state.

## Submission record

Prediction / expected case:

Actual result and command:

Failure case and repair:

Changed-case explanation:

Reviewer/self-check and remaining gap:

# Module 02 integration project: A bounded five-turn resource game

Build a terminal resource simulator with health in 0..10 and resources in 0..9. Start at health=10 and resources=0. For this module project, each of five supplied turns consumes one turn slot, even when the request cannot be fulfilled. This is a deliberately specified project rule; it differs from the later capstone's rejected-movement accounting.

The command collect adds three resources up to capacity and removes one health down to zero. Rest restores two health up to ten. Buy spends five resources only when affordable and otherwise changes neither health nor resources. Unknown text changes neither value and reports the problem. Stop the five-turn run early if health reaches zero. Keep the transition function separate from prompting and printing.

First use the fixed commands collect, collect, buy, rest, buy. Predict all five results before running. Then offer interactive input and show invalid-input behavior without allowing negative state.

**Acceptance:** The fixed sequence ends with health 10 and resources 1; the full trace is retained. Test caps, zero turns in a reusable driver, invalid text, exact affordability and unaffordable purchase. Explain the project-specific turn policy rather than borrowing one accidentally.

## Ten additional exercises

1. **M02-E01.** Predict "8"+"2", int("8")+int("2") and int("8")*int("2").
2. **M02-E02.** Add a count ceiling of 99 to a copied calculator. Choose three boundary inputs.
3. **M02-E03.** Compare two key-door groupings using a key holder with stamina zero.
4. **M02-E04.** Explain why level>5 is not equivalent to level>=5.
5. **M02-E05.** Trace three resource turns with the collect-three/spend-five rule.
6. **M02-E06.** Change collection to seven and explain which invariant survives.
7. **M02-E07.** Write a while-loop example that never changes the condition, then describe the repair without running an infinite loop.
8. **M02-E08.** Design two callers for advance: one prints and one tests.
9. **M02-E09.** Test a movement with x=10, velocity=-4, dt=0.5.
10. **M02-E10.** Show how a caller preserves a previous position after advance rejects dt.

## Five-question module review

Each answer earns 0-2 points. Suggested pass 8/10 plus project evidence.

1. **M02-R01.** What is the distinction between parsing text and accepting a game-domain value?
2. **M02-R02.** How many local door-boundary combinations use two key flags and three values for each numeric threshold?
3. **M02-R03.** Why does nonnegative inventory alone not prove a loop ran the correct number of turns?
4. **M02-R04.** What does return allow that printing alone does not?
5. **M02-R05.** For the five-turn project sequence, explain the last rejected buy.

**Project evidence:** source and inputs, test results, known limits, changed requirement, review decision.

# Module 03: Organizing data and small programs

**Prerequisites:** Module 02.

## Lab 03.1: Sequences, dictionaries and identity

**Goal:** Choose collections according to the operations the program needs.

Create five flat unit records and implement `index_units` and `living_units` in `labs/student/lab_03_1.py`. Use stable IDs, reject duplicate IDs and return independent shallow copies of flat records. Preserve the input collection while filtering.

Test an empty collection, all-dead units, duplicate names with different IDs and duplicate IDs. Rename a copied living unit and demonstrate that the original record is unchanged. Explain why the same technique would not isolate a nested inventory list. Keep the small input data and the before/after snapshots in your journal.

**Changed input:** Add a nested list of item IDs in a separate experiment and demonstrate the limit of a shallow record copy. Document one design that would make ownership clear.

### Function interfaces

`index_units(units)`: Flat unit records with id/name/health; duplicate IDs are rejected. Return a dictionary of independent shallow record copies.

`living_units(units)`: Return new flat-record copies for units with health > 0.

### Submission record

Prediction / expected case:

Actual result and command:

Failure case and repair:

Changed-case explanation:

Reviewer/self-check and remaining gap:

# Lab 03.2: Decomposition, modules and interfaces

**Goal:** Divide a small game into parts that can be tested independently.

Implement the strict `spend` contract in `labs/student/lab_03_2.py`. Refactor your earlier resource simulator into a rule module and a separate command-line entry point in `workspace/module03_simulator/`. The rule module must be importable without output or prompts.

Save a before-refactor trace and compare it with the after-refactor trace. Add direct function tests for exact affordability, insufficient balance, zero cost and invalid input. Pass by drawing the dependency direction and showing an independent test that uses the rule without launching the interface.

**Changed input:** Create a second caller that feeds a fixed list of requests without user input. It should reuse the same rule module rather than copy its arithmetic.

## Function interfaces

`spend(resources, cost)`: Nonnegative integers; unaffordable spending leaves resources unchanged.

## Submission record

Prediction / expected case:

Actual result and command:

Failure case and repair:

Changed-case explanation:

Reviewer/self-check and remaining gap:

# Lab 03.3: Files, serialization and validation

**Goal:** Save structured data without assuming every file is trustworthy.

Implement `encode_save` and `decode_save` in `labs/student/lab_03_3.py`, including the exact schema described above, duplicate-key rejection and the input-length limit. Return new dictionaries and do not write files inside these functions. Use a separate disposable workspace file to exercise a file wrapper.

Test a valid round trip, missing and extra fields, wrong types, a Boolean version, unsupported version, out-of-range health, duplicate keys, excessive text and truncated input. Keep the known-good state and save file unchanged after a failed load. Pass only after explaining why successful JSON parsing is not the same as valid game data.

**Changed input:** Propose a version-2 save with a new field. Write the policy for rejecting or migrating version 1 before implementing any migration.

## Function interfaces

`encode_save(state)`: Validate then serialize one small flat save; never writes a file.

`decode_save(text)`: At most 10,000 text characters; validate before returning a new state.

## Submission record

Prediction / expected case:

Actual result and command:

Failure case and repair:

Changed-case explanation:

Reviewer/self-check and remaining gap:

# Lab 03.4: Classes, composition and state machines

**Goal:** Represent behavior without creating an unnecessary inheritance hierarchy.

Implement `transition` in `labs/student/lab_03_4.py` with the exact five-entry table and the stated no-op/error policy. Build a small controller wrapper in your workspace and add an inventory field by composition rather than a hierarchy of movement subclasses.

Test every allowed transition, at least three forbidden transitions and an unknown state. Create two controller instances and demonstrate independence. Trace move, hit, move, recover, move, stop from idle. Pass by explaining both the state table and which object or function owns each change.

**Changed input:** Add an independent inventory operation while the controller is stunned. Decide explicitly whether it is allowed; do not let inheritance structure make the game-design decision accidentally.

## Function interfaces

`transition(state, event)`: Unknown state is an error; an unlisted event leaves a valid state alone.

## Submission record

Prediction / expected case:

Actual result and command:

Failure case and repair:

Changed-case explanation:

Reviewer/self-check and remaining gap:

# Module 03 integration project: A data-driven simulator with validated persistence

Refactor the resource game into a rule module, command-line entry point and persistence module. Store five flat entity records with stable IDs; at least one represents the player and the others may be nonacting units for lookup/filter exercises. Reject duplicate IDs and document whether indexes contain references or copies.

Define a small versioned configuration file controlling starting health, health cap, starting resources, capacity, collection amount and purchase cost. Write its exact schema before parsing it. Validate integer types, nonnegative amounts and relationships such as starting_resources <= capacity. Do not require students to guess a schema from source code.

Implement save/load with a matching explicit state schema and a supported version policy. Keep a known-good state available during failed loading. Integrate the three-state controller from 03.4 as a separately testable component; inventory must not depend on a subclass for each movement mode.

**Acceptance:** A clean run reads valid configuration, accepts a saved round trip, rejects malformed/unsupported data and preserves live state after failure. Imports do not prompt. A changed unit name does not invalidate ID lookup. Demonstrate all five allowed controller transitions and three forbidden ones.

## Ten additional exercises

1. **M03-E01.** Create two units with the same name but distinct IDs and show stable lookup.
2. **M03-E02.** Demonstrate outer-list copying without record copying.
3. **M03-E03.** Demonstrate why a flat-record copy does not isolate a nested inventory list.
4. **M03-E04.** Filter five units without deleting from the source during iteration.
5. **M03-E05.** Draw the dependency direction between interface and simulation.
6. **M03-E06.** Create a syntactically valid JSON save that fails the game schema.
7. **M03-E07.** Explain why true is rejected as version 1 by the strict schema.
8. **M03-E08.** Specify unknown-field behavior for a future configuration version.
9. **M03-E09.** Trace move, hit, move, recover, move, stop from idle.
10. **M03-E10.** Create two controllers and prove one can move without changing the other.

## Five-question module review

Each answer earns 0-2 points. Suggested pass 8/10 plus project evidence.

1. **M03-R01.** Why is a list index usually a weak long-lived entity identifier?
2. **M03-R02.** What does the main guard prevent, and what does it not prevent?
3. **M03-R03.** Why is successful JSON parsing insufficient for loading a game?
4. **M03-R04.** What happens to live state when candidate decoding fails before assignment?
5. **M03-R05.** Why use composition for inventory and movement state?

**Project evidence:** source and inputs, test results, known limits, changed requirement, review decision.

# Module 04: Debugging, testing and collaboration

**Prerequisites:** Modules 01-03.

## Lab 04.1: Debugging as an experiment

**Goal:** Find a defect by narrowing evidence rather than making random edits.

Introduce a double-damage bug into a disposable copy of your simulator. Write a minimal report with health 10, damage 3, expected 7 and actual behavior. Trace or break at the first incorrect update. Add a failing test before repairing the copied simulator.

Implement `apply_damage` in `labs/student/lab_04_1.py`: nonnegative integer inputs, one subtraction and a zero floor. Demonstrate that the repaired version passes, and that temporarily restoring the known bug makes a relevant test fail. The supplied `tools/mutation_demo.py` illustrates this method; your own bug report and regression test remain required.

**Changed input:** Construct an integration bug where a correct damage function is called twice. Explain why fixing the arithmetic function would be the wrong repair.

### Function interfaces

`apply_damage(health, damage)`: Apply one nonnegative integer damage event; health cannot fall below zero.

### Submission record

Prediction / expected case:

Actual result and command:

Failure case and repair:

Changed-case explanation:

Reviewer/self-check and remaining gap:

# Lab 04.2: Unit, integration and property tests

**Goal:** Choose tests that challenge the contract instead of repeating the implementation.

Implement `bounded_move` in `labs/student/lab_04_2.py` with the documented whole-move rejection policy. The starting size must be positive and the starting position inside the board. Then write at least twelve learner-owned tests in `workspace/test_my_rules.py` across motion, spending and save/load, including at least one bounded property loop.

Record each test's requirement and independently derived expected result. Introduce an off-by-one or double-update mutation in a disposable copy and show a named test detects it. Restore the correct code. A complete submission includes tests, a successful run and the mutation experiment, not only the supplied checker output.

**Changed input:** Write a deliberately useless implementation that never moves. Identify which property tests it passes and which expected-transition test rejects it.

## Function interfaces

`bounded_move(position, delta, size)`: Trusted integer inputs; position within [0,size). Reject out-of-bounds move.

## Submission record

Prediction / expected case:

Actual result and command:

Failure case and repair:

Changed-case explanation:

Reviewer/self-check and remaining gap:

# Lab 04.3: Git, branches and recoverable changes

**Goal:** Use small commits to preserve reasoning and collaborate safely.

Use the Git worksheet in `templates/GIT_LAB.md` inside a new disposable practice directory. Make three focused commits, implement a change on a branch, merge it and revert an intentionally bad commit. Before committing, inspect staged differences. Save the log, final rules, commands and an explanation of the staging area in `workspace/04_3_git_evidence.md`.

Run the supplied sandbox demonstration as a comparison, not as a substitute for your own exercise. Pass by identifying the exact content of each commit, explaining why the revert adds history and showing that your working tree is clean after completion.

**Changed input:** Stage a file, edit it again, then compare git diff with git diff --staged. Explain which version would enter the next commit.

**Check type:** evidence-based review, not an automated function test.

## Submission record

Prediction / expected case:

Actual result and command:

Failure case and repair:

Changed-case explanation:

Reviewer/self-check and remaining gap:

# Lab 04.4: Review, documentation and responsible AI help

**Goal:** Explain a change and verify assistance without surrendering understanding.

Review a roughly ten-line change using `templates/CODE_REVIEW.md`. Address requirement, correctness, boundaries, side effects, clarity and tests. Produce a README for your simulator that another person can follow from a clean folder. Record assistance and provenance in the template.

Perform one small related change without copying the reference, predict its effects and update the tests. This is an evidence-based lab; there is no automatic function checker for it. Pass by explaining every changed line and by recording a clean-start result from another person or a clearly labeled self-review when independent review is not yet available.

**Changed input:** Review a change that moves input() into a previously pure function. Explain the interface regression even when normal interactive play still appears to work.

**Check type:** evidence-based review, not an automated function test.

### Submission record

Prediction / expected case:

Actual result and command:

Failure case and repair:

Changed-case explanation:

Reviewer/self-check and remaining gap:

# Module 04 integration project: A reviewed, tested local release

Create a locally tagged release of the resource simulator. Supply a minimal bug report, a failing-then-passing regression test, at least twelve learner-written tests, a controlled mutation experiment, a reviewed change and an executable README. A remote public upload is not required.

Keep the work in a local Git repository, using focused commits and a feature branch. Practice reverting a bad change in a disposable copy or separate practice repository. Record the environment and exact test command. Review staged files for secrets and unrelated content before committing.

Ask a reviewer to trace a roughly ten-line change and follow the README in a clean folder. When an independent reviewer is unavailable, record self-review honestly and keep the independent check as an open completion item.

**Acceptance:** The supplied defect is reproduced before its fix; a named test catches its reintroduction; the restored implementation passes the learner suite. Git history explains the changes. The README works from a clean extraction, and the learner can implement a related variation without copying.

## Ten additional exercises

1. **M04-E01.** Write a minimal double-damage report with one player and one event.
2. **M04-E02.** Create a correct function called twice by a buggy dispatcher.
3. **M04-E03.** Write a bounds test that fails an <=size mutant.
4. **M04-E04.** Write a progress test rejecting an always-no-op movement function.
5. **M04-E05.** Design a temporary-file test that never touches personal data.
6. **M04-E06.** Write one isolated test that does not rely on a prior test initializing state.
7. **M04-E07.** Stage one file version, edit it, and compare staged and unstaged differences.
8. **M04-E08.** Explain a revert using a four-commit history.
9. **M04-E09.** Review exact-affordability and zero-cost boundaries in a purchase change.
10. **M04-E10.** Write a provenance note for assisted code.

## Five-question module review

Each answer earns 0-2 points. Suggested pass 8/10 plus project evidence.

1. **M04-R01.** Why find the earliest state disagreement rather than only inspecting the final screen?
2. **M04-R02.** What is an independent test oracle in this setting?
3. **M04-R03.** What must a mutation experiment demonstrate?
4. **M04-R04.** Does git add always include edits made afterward?
5. **M04-R05.** What makes a README executable rather than merely descriptive?

**Project evidence:** source and inputs, test results, known limits, changed requirement, review decision.

# Module 05: A complete first game

**Prerequisites:** Modules 01-04.

## Lab 05.1: The update loop and command model

**Goal:** Separate simulation ticks from input collection and presentation.

Implement `normalise_command`, `step_position` and `render_grid` in `labs/student/lab_05_1.py`. The default board is 10x10; the helpers also accept a small square size for tests. Draw P at the player and E at the bottom-right exit unless the player occupies it. Unknown or out-of-bounds movement returns the unchanged valid position.

Connect these helpers in a terminal adapter in your workspace. Test one movement per input and repeated drawing with no updates. Compare a known 3x3 rendering, then run the actual 10x10 version. Pass by showing that the update can be called without input or printing and that the display is read-only.

**Changed input:** Add a second read-only display function that reports numeric coordinates. Calling both displays must still produce only one update per command.

### Function interfaces

`normalise_command(text)`: Text command -> stripped lowercase text.

`step_position(position, command, size)`: Valid integer tuple position; known legal move or unchanged position.

`render_grid(position, size)`: Return exactly size rows; P is the player; E is the bottom-right exit; other cells are dots.

### Submission record

Prediction / expected case:

Actual result and command:

Failure case and repair:

Changed-case explanation:

Reviewer/self-check and remaining gap:

# Lab 05.2: Collision rules and complete behavior

**Goal:** Specify edge cases before implementing a playable rule.

Implement `step_world` in `labs/student/lab_05_2.py` with the fixed map, pure flat-state return and terminal win policy. Test all four edges, all five obstacles, a normal move, reaching the goal and commands after victory. Preserve the input dictionary and reject the complete move rather than changing one coordinate early.

Once the helper passes, implement the capstone energy/loss/restart requirements from `CAPSTONE_SPEC.md` in your own game. Keep separate evidence for the helper and the richer game. Pass by explaining collision rejection, terminal priority and a live restart that leaves no stale round state.

**Changed input:** Reach the goal on exactly the last energy unit by waiting seven times, moving right nine times and down nine times. Predict the final status before testing.

## Function interfaces

`step_world(world, command)`: Valid flat {x,y,won} state on a fixed 10x10 map; no mutation. Obstacles and edges reject the whole move. Once won, all commands no-op. This teaching model omits the capstone's energy and restart rules.

## Submission record

Prediction / expected case:

Actual result and command:

Failure case and repair:

Changed-case explanation:

Reviewer/self-check and remaining gap:

# Lab 05.3: Deterministic replay and simple measurement

**Goal:** Reproduce a run and measure the work actually performed.

Complete `labs/student/lab_05_3.py` after the Lesson 05.2 helper works. Replay an empty session, a twenty-command session and the known route to the goal. Compare both state fields and canonical digests across repeated runs. Preserve exact commands and rule assumptions.

Run `tools/benchmark_game.py` as a reference experiment, then adapt the same method to your own implementation without comparing unlike workloads. Keep the raw JSON report and state which operations are included. Pass by reproducing a session and explaining why the microbenchmark is not a whole-game FPS measurement.

**Changed input:** Benchmark a deliberately poor workload that spends almost all iterations in a terminal state. Compare its timed operation with the active-play cycle before comparing durations.

## Function interfaces

`replay(commands)`: Replay on the Lesson 05.2 map from the fixed initial state.

`state_digest(state)`: Canonical serialization digest for comparison, not a correctness proof.

## Submission record

Prediction / expected case:

Actual result and command:

Failure case and repair:

Changed-case explanation:

Reviewer/self-check and remaining gap:

# Lab 05.4: Finishing, usability and the first portfolio artifact

**Goal:** Deliver a tiny game another person can run and understand.

Complete `CAPSTONE_SPEC.md` and the five-module project requirements. Build your own game under `capstone/student/` or an explicitly documented workspace. Replace the intentionally failing starter test with at least twelve independent tests and add a meaningful design change rather than merely submitting the reference.

Ask a tester to follow your README without coaching, record three observations and repair at least one issue. Submit the release evidence and demonstrate a live rule variation. The supplied game checker tests the reference, not your project; use your own test command and independent review for Gate A.

**Changed input:** Prepare a handoff note explaining one known limitation, one tested failure path and one proposed next feature, each clearly separated from claims of completed work.

**Check type:** evidence-based review, not an automated function test.

### Submission record

Prediction / expected case:

Actual result and command:

Failure case and repair:

Changed-case explanation:

Reviewer/self-check and remaining gap:

# Module 05 integration project: Training Yard: the Gate A first-game release

Implement the complete first game described in CAPSTONE_SPEC.md. Build on the pure movement and collision helpers but add the explicitly specified energy, status, restart, save/load, replay and interface policies. Do not import the supplied reference implementation into the student project.

The reference is a working comparison with its own test suite. It is not the learner's finished capstone. A submission must include meaningful independently implemented behavior, at least twelve learner-authored tests, a reproducible twenty-command replay and a small, accurately scoped performance experiment.

Complete a clean-start review, record three tester observations and improve at least one usability problem. Deliver a rule specification, source, tests, README, two-bug journal, assistance record and a live-change demonstration.

**Acceptance:** All essential Gate A rules pass the student-owned tests and manual review. The tester can start, play, encounter rejection, win or lose, restart, save/load, export/replay, quit and rerun tests. A copied reference plus its test log does not pass.

## Ten additional exercises

1. **M05-E01.** Write the four coordinate deltas for the declared screen-grid convention.
2. **M05-E02.** Show two render calls with no intervening update and compare state.
3. **M05-E03.** Construct a blocked movement at each edge.
4. **M05-E04.** Construct a wall collision that would expose partial energy deduction.
5. **M05-E05.** Predict the known win route: nine d then nine s.
6. **M05-E06.** Predict seven waits followed by the win route.
7. **M05-E07.** Explain how a successful load affects recording in the reference interface.
8. **M05-E08.** Compare canonical state serialization with a screenshot.
9. **M05-E09.** Design a benchmark avoiding terminal-state no-op dominance.
10. **M05-E10.** Propose a meaningful personal extension and its regression risks.

## Five-question module review

Each answer earns 0-2 points. Suggested pass 8/10 plus project evidence.

1. **M05-R01.** Which fields remain unchanged on a rejected wall movement in the capstone?
2. **M05-R02.** How is the win-versus-exhaustion tie resolved?
3. **M05-R03.** What belongs in a reproducible replay besides commands?
4. **M05-R04.** Why is the supplied benchmark not a frames-per-second result?
5. **M05-R05.** What distinguishes a finished student artifact from a supplied demonstration?

**Project evidence:** source and inputs, test results, known limits, changed requirement, review decision.